Software Architecture Laboratory

Laboratory No. 6
BluePrints React UI

React 18 + Redux Toolkit + Axios + JWT + Vitest

Students:

Andersson David Sanchez Mendez

Cristian Santiago Pedraza Rodriguez

Elizabeth Correa Suarez

Juan Sebastian Ortega Munoz

Instructor: Professor Javier Ivan Toquica Barrera

Course: Software Architecture (ARSW)

Institution: Escuela Colombiana de Ingenieria Julio Garavito

March 2026

0 Contents

1 Objective **3**

2 Technology Stack and Project Structure **3**
2.1 Stack Overview . . . 3
2.2 Directory Layout . . . 3

3 Architecture and Data Flow **3**
3.1 Frontend Architecture . . . 3
3.2 JWT Authentication Flow . . . 3

4 Functional Requirements Implementation **3**
4.1 Requirement 1: Canvas Rendering and Interaction . . . 3
4.2 Requirement 2: Author-Based Blueprint Listing . . . 4
4.3 Requirement 3: Open Blueprint and Update Canvas . . . 4
4.4 Requirement 4: Service Switching (Mock/API) . . . 4
4.5 Requirement 5: Redux-Driven UI State . . . 4
4.6 Requirement 6: UI/UX Styling . . . 4
4.7 Requirement 7: Testing and Quality . . . 5

5 Blueprint Create/Update Reliability Improvements **5**

6 Build, Scripts, and Tooling **5**
6.1 NPM Commands . . . 5

7 Quality Metrics **6**

8 Synthesis of Recommended Readings **6**
8.1 React Best Practices (freeCodeCamp) . . . 6
8.2 State Management in React (freeCodeCamp + Redux Toolkit article) . . . 6
8.3 Axios Interceptors Documentation . . . 6

9 Conclusions **6**

10 References **7**

1 Objective

This laboratory delivers a production-oriented Single Page Application (SPA) for the BluePrints ecosystem, modernizing the original static client into a React architecture with robust state management, JWT-aware HTTP communication, canvas interaction, and test-driven quality controls.

Primary Learning Objectives

1. Build a modular React UI with reusable components and route-based views.
2. Implement global state using Redux Toolkit slices and async thunks.
3. Integrate JWT authentication with Axios interceptors and protected routes.
4. Support author lookup, blueprint visualization, point editing, and persistence.
5. Provide service abstraction (real API vs in-memory mock) using environment flags.
6. Achieve high automated test confidence with Vitest + Testing Library.

2 Technology Stack and Project Structure

2.1 Stack Overview

Technology	Role in the Solution
React 18	Component-based SPA rendering
React Router 6	Navigation and route guards
Redux Toolkit	Global state and async API orchestration
Axios	HTTP client with JWT interceptor
Vite	Development/build tooling
Vitest + Testing Library	Unit/component testing
ESLint + Prettier	Code quality and formatting

2.2 Directory Layout

```
src/
  components/    # BlueprintCanvas, BlueprintForm,
       BlueprintList, PrivateRoute
  features/      # Redux slice: blueprintsSlice.js
  pages/         # BlueprintsPage, LoginPage,
       ProfilePage, ChangePasswordPage
  services/      # apiClient, blueprintsService,
       apimock, auth helpers
  store/         # Redux store configuration
  App.jsx
tests/           # 21 test files + setup.js
images/          # Laboratory evidence screenshots
report/          # This LaTeX report
```

Listing 1: Frontend repository structure (simplified)

3 Architecture and Data Flow

3.1 Frontend Architecture

The application adopts a layered frontend architecture:

1. **Presentation Layer:** Pages and reusable components.
2. **State Layer:** Redux slice with async thunks and reducers.
3. **Service Layer:** API adapter that normalizes backend response envelopes.
4. **Transport Layer:** Axios instance with JWT injection and auth handling.

Key Design Decision

The backend security API returns envelope payloads (for example `{code, message, data}`). The frontend service normalizes these responses before they reach Redux. This decision avoids duplicating response-shape checks in components and reducers.

3.2 JWT Authentication Flow

1. User signs in at `/login`.
2. UI posts credentials to `/auth/login`.
3. `access_token` is stored in `localStorage`.
4. Axios interceptor appends `Authorization: Bearer <token>`.
5. Protected routes become accessible through `PrivateRoute`.

```
1 api.interceptors.request.use((config) => {
2   const token = localStorage.getItem('token')
3   if (token) config.headers.Authorization = `Bearer
        ${token}`
4   return config
5 })
```

Listing 2: Axios interceptor strategy (simplified)

4 Functional Requirements Implementation

4.1 Requirement 1: Canvas Rendering and Interaction

The `BlueprintCanvas` component renders blueprint points and polylines, applies viewport scaling, and supports click-to-add-point interaction for the currently opened blueprint.

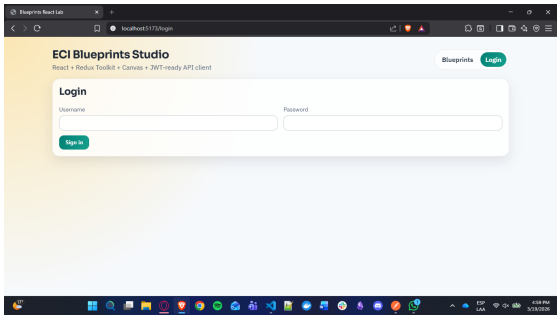

Figure 1: Canvas rendering for blueprint visualization

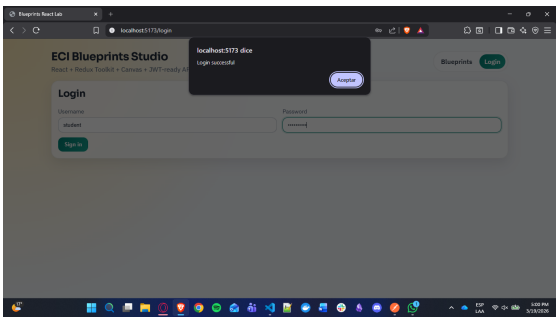

Figure 2: Canvas available after successful authentication

4.2 Requirement 2: Author-Based Blueprint Listing

Users query an author and receive a table with blueprint names, point counts, and an Open action to load blueprint details.

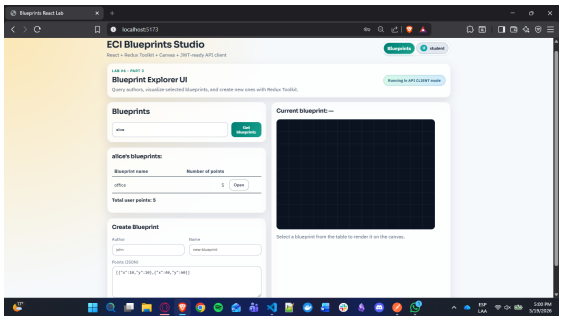

Figure 3: Author query and blueprints table

4.3 Requirement 3: Open Blueprint and Update Canvas

Selecting Open synchronizes Redux `current` blueprint and redraws the canvas immediately.

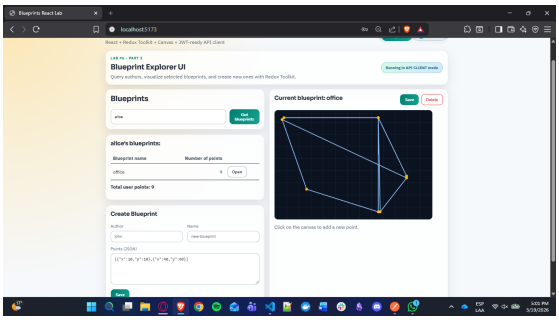

Figure 4: UI state and canvas synchronized after selecting a blueprint

4.4 Requirement 4: Service Switching (Mock/API)

The system supports a runtime switch controlled by `VITE_USE_MOCK`. Both service implementations expose the same interface.

```
const useMock = String(import.meta.env.
    VITE_USE_MOCK).toLowerCase() === 'true'
const service = useMock ? apimock : apiclient
```

```
export default service
```

Listing 3: Service selector pattern (conceptual)

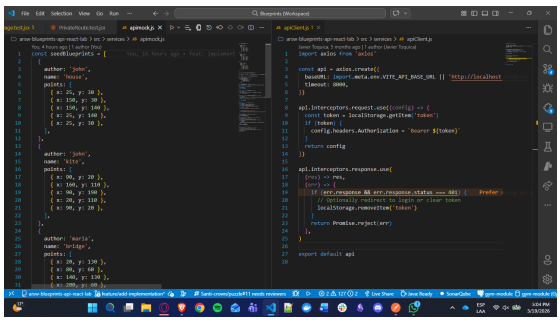

Figure 5: Evidence of environment-driven service switching

4.5 Requirement 5: Redux-Driven UI State

The page displays current blueprint metadata and aggregated points using `useSelector` and memoized derivations.

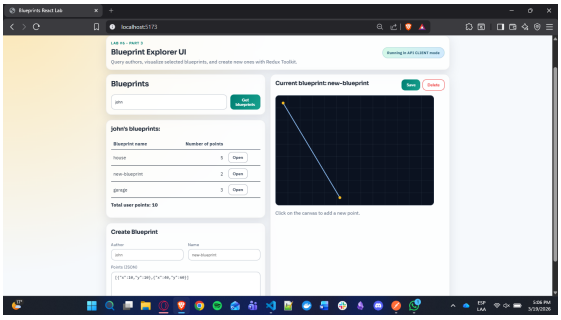

Figure 6: Redux-managed state reflected in the UI

4.6 Requirement 6: UI/UX Styling

The interface uses custom styling (cards, gradients, spacing, responsive grid, state color semantics) and includes profile/password management views.

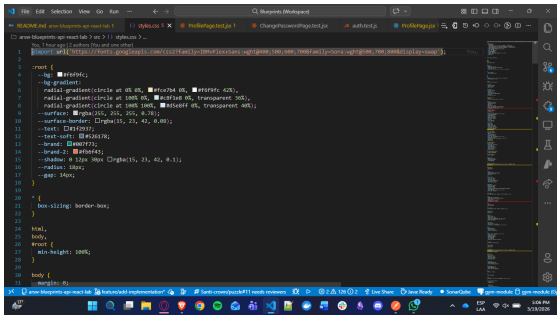

Figure 7: Main colorful dashboard design

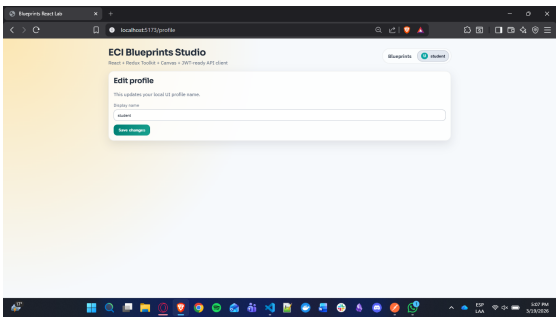

Figure 8: Profile page styling and interaction

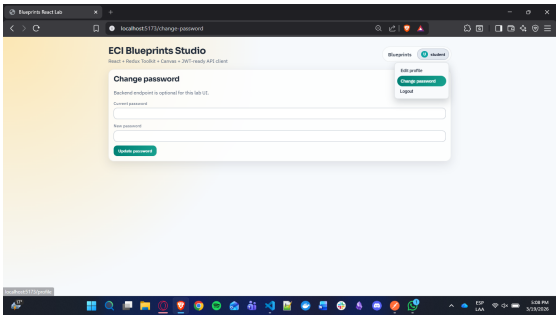

Figure 9: Change password page styling and interaction

4.7 Requirement 7: Testing and Quality

The project includes reducer tests, component tests, routing tests, service tests, auth utility tests, and integration-like UI flow tests.

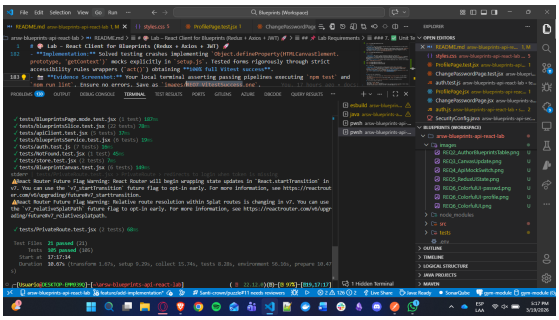

Figure 10: Vitest execution evidence

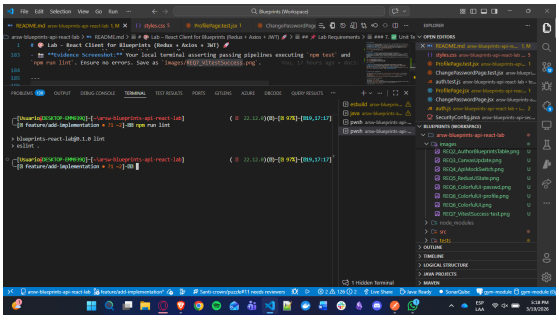

Figure 11: ESLint validation evidence

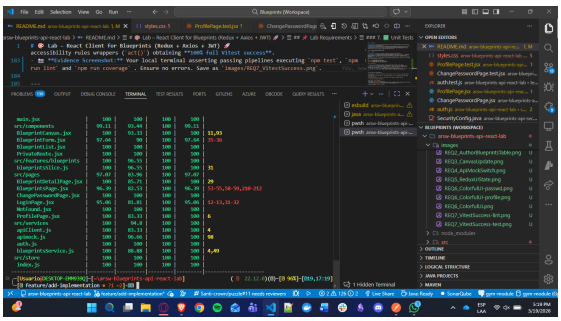

Figure 12: Coverage execution evidence

5 Blueprint Create/Update Reliability Improvements

During integration against the secured Java backend, key reliability fixes were implemented to align frontend write operations with the backend contract.

Implemented Stability Fixes

- Added response-envelope unwrapping in service layer to support `{code, message, data}` and `{status, message, data}`.
- Updated save flow to persist only newly created points through the backend endpoint `/blueprints/{author}/{name}/points`.
- Added duplicate prevention and clearer conflict feedback for blueprint creation.
- Added defensive point JSON parsing/normalization in creation form.
- Expanded regression tests for service and page-level create/save flows.

6 Build, Scripts, and Tooling

6.1 NPM Commands

Script	Purpose
`npm run dev`	Starts Vite development server
`npm run build`	Produces production bundle
`npm run preview`	Runs preview server for built bundle
`npm run lint`	Runs ESLint checks
`npm run format`	Applies Prettier formatting
`npm test`	Executes Vitest in run mode
`npm run coverage`	Generates test coverage report

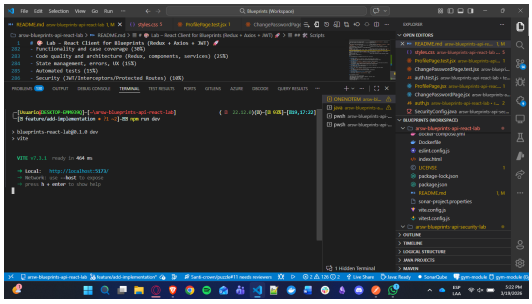

Figure 13: Development server execution

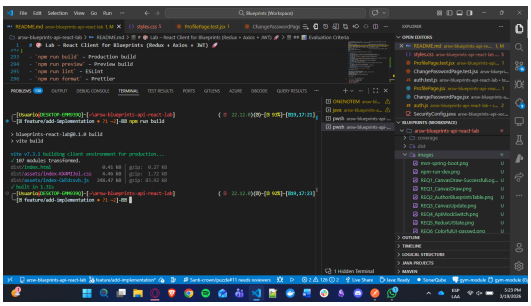

Figure 14: Production build execution

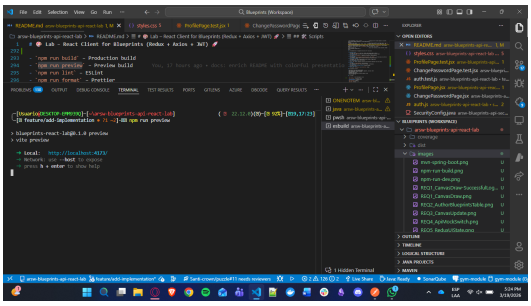

Figure 15: Preview execution

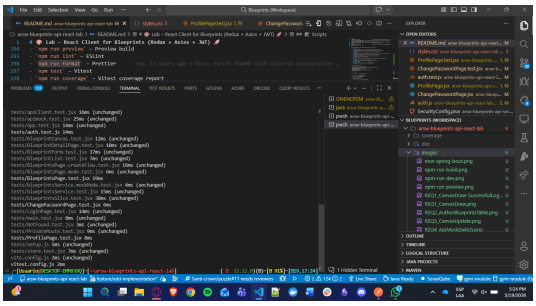

Figure 16: Formatting workflow

7 Quality Metrics

Metric	Observed Value
Passing tests	105 / 105
Statements coverage	99.63%
Branches coverage	95.81%
Functions coverage	97.14%
Lines coverage	99.63%

Verification Snapshot

- Unit and integration-style frontend tests pass successfully.
- Lint and build pipelines execute without blocking errors.
- JWT auth flow is validated in UI and backend-integrated paths.
- Mock/API parity is preserved through shared service contracts.

8 Synthesis of Recommended Readings

This section summarizes actionable lessons extracted from the requested references and their direct application in this project.

8.1 React Best Practices (freeCodeCamp)

- Keep components focused and reusable.
- Organize by feature and concern, not by file extension only.
- Treat side effects explicitly and isolate API calls in dedicated layers.
- Avoid ad-hoc global mutable state.

Applied in this lab: separate folders for components/pages/services, clear route boundaries, isolated API service modules, and reusable UI blocks.

8.2 State Management in React (freeCodeCamp + Redux Toolkit article)

- Escalate from local state to centralized state only when needed.
- Use predictable reducers and immutable updates.
- Use async abstraction patterns for network-dependent transitions.

Applied in this lab: `createSlice` + `createAsyncThunk`, single source of truth for blueprint state, and structured loading/error flows.

8.3 Axios Interceptors Documentation

- Interceptors are ideal for cross-cutting concerns (auth headers, centralized response handling).
- Keep auth token injection close to transport concerns.

Applied in this lab: request interceptor appends Bearer JWT globally; response interceptor handles invalid-session scenarios.

9 Conclusions

Key Findings

1. The migration from a static client to a React SPA significantly improves maintainability, traceability, and extensibility.
2. Redux Toolkit provides a balanced state model for medium-size frontend systems where asynchronous workflows dominate.
3. Service abstraction plus response normalization is critical when backend response contracts evolve during iterative labs.
4. JWT integration with Axios interceptors and protected routes creates a clean and scalable authentication boundary in the UI.
5. High automated test coverage helped detect regressions early while implementing advanced requirements and UX refinements.

Note

Future extension opportunities include RTK Query migration for request caching, MSW-based network simulation for deterministic test isolation, and CI pipelines that publish coverage artifacts for each pull request.

10 References

1. freeCodeCamp. Best Practices for React. https://www.freecodecamp.org/news/best-practices-for-react/
2. freeCodeCamp. How to Manage State in a React App. https://www.freecodecamp.org/news/how-to-manage-state-in-a-react-app/
3. Axios Documentation. Interceptors. https://axios-http.com/docs/interceptors
4. Aysu Nita. Mastering State Management in React with Redux and Redux Toolkit. https://medium.com/@aysunitai/mastering-state-management-in-react-with-redux-and-redux-toolkit-d8e6f09d5393
5. React Documentation. https://react.dev/
6. Redux Toolkit Documentation. https://redux-toolkit.js.org/
7. React Router Documentation. https://reactrouter.com/
8. Vitest Documentation. https://vitest.dev/
9. Testing Library Documentation. https://testing-library.com/docs/
10. Vite Documentation. https://vitejs.dev/